Molfig

Static molecular structure rendering for Typst

v0.1.4 2026-08-26 MIT

Render molecular structures through maquette.

Eito Yoneyama

Molfig turns PDB, mmCIF, BinaryCIF, and XYZ structures into static OBJ, STL, or PLY meshes inside a Rust WebAssembly plugin, then hands those meshes to maquette for document rendering.

Table of Contents

Part I

Overview

Molfig is a Typst package for molecular figures in static documents. It accepts PDB, mmCIF, BinaryCIF, and XYZ input, converts the structure into a CPU-side Mol*-style Model/Structure/Unit representation, builds static geometry, exports the mesh as OBJ, STL, or PLY, and delegates final rendering to maquette.

Mol* is an open-source molecular visualization toolkit and web viewer. Molfig uses Mol* as the compatibility target for structure interpretation, static geometry, and OBJ/STL export behavior; see molstar.org[1] and molstar/molstar[2].

I.1 Design Goals

Molfig is designed around four constraints:

1. Typst packages cannot invent paths into the caller's project. On Typst 0.15.0 or later, the caller may pass a project-side `path(...)` value and Molfig will read it as bytes. For Typst 0.14-compatible documents, the caller should pass bytes produced by `read(..., encoding: none)`.
2. The plugin should run in Typst's WebAssembly environment without WebGL.
3. The molecular path should preserve Mol*-style structure concepts rather than flattening everything at parse time.
4. The final image should be a document-native static rendering through maquette, not an interactive viewer snapshot.

> Molfig generates static meshes and is not a Mol* WebGL renderer. `representation: "surface"` computes the Mol* molecular-surface field and marching-cubes mesh on the CPU. Gaussian volume and density-map visuals remain outside the static export contract; ViewerAuto's Huge and Gigantic Gaussian surfaces are likewise generated on the CPU.

I.2 Rendering Pipeline

Stage	Control	Role
Read	`path(...)` or `read(..., encoding: none)`	The user document supplies either a Typst 0.15+ project path or already-read structure bytes.
Parse	`format`	The plugin parses PDB, text CIF/mmCIF, BinaryCIF MessagePack, or XYZ coordinate data.
Model	`assembly`, `altloc`	Model, Structure, Unit, unit operators, altLoc policy, bonds, secondary structure, and coarse data are selected.

[1] https://molstar.org/
[2] https://github.com/molstar/molstar

Geometry	`representation`, `quality`	The static geometry layer builds spheres, cylinders, tubes, sheets, ribbons, nucleotide visuals, carbohydrate visuals, and related mesh spans.
Export	`mesh-format`	The mesh is serialized as OBJ, binary STL, or ASCII PLY bytes.
Render	`config`	The exported bytes and maquette configuration are passed to maquette.

Part II

Quickstart

The following example renders PDB entry 9R1O. Put `9R1O.typ` and `9R1O.pdb` in the same directory, then compile the Typst file.

Complete 9R1O example — *9R1O.typ*

```
#import "@preview/molfig:0.1.4"

#set page(width: auto, height: auto, margin: 0mm)

// Uses structural data from RCSB PDB / wwPDB.
// PDB ID: 9R1O
// PDB DOI: https://doi.org/10.2210/pdb9R1O/pdb
// PDB archive data files are available under CC0 1.0.
#let pdb = path("9R1O.pdb")

#molfig.render(
  pdb,
  format: "pdb",
  representation: "cartoon",
  assembly: "1",
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "svg",
  config: (
    azimuth: 35,
    elevation: 24,
    background: "",
  ),
)
```

The `path("9R1O.pdb")` input in this example requires Typst 0.15.0 or later. On an older Typst version, replace it with `read("9R1O.pdb", encoding: none)` and pass the resulting bytes to `molfig.render`.

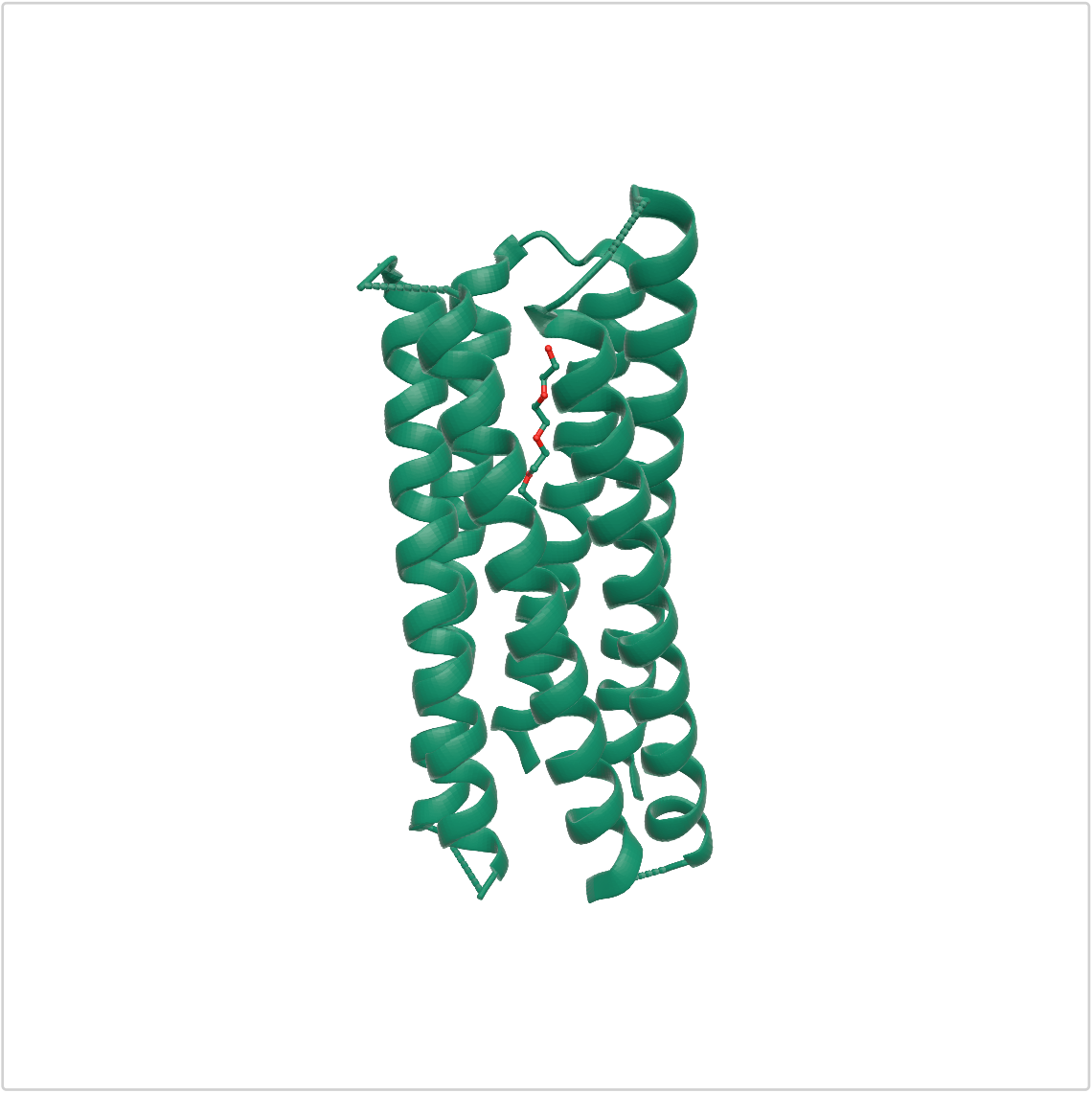

Figure 1: RCSB PDB entry 9R1O rendered by Molfig with the complete Quickstart settings.

Structural data source: RCSB PDB / wwPDB, PDB ID `9R10`, https://doi.org/10.2210/pdb9R1O/pdb[3]. PDB archive data files are available under CC0 1.0. Deposition authors: Petrenas, Ozga, Chubb, and Woolfson.

II.1 Reading Structure Files

On Typst 0.15.0 or later, prefer passing a `path(...)` value for file-backed structure data. The path is resolved by Typst relative to the caller's file, and Molfig reads it internally with `encoding: none`. This keeps package calls concise while preserving the caller-side project boundary.

Format	Typst 0.15+ path	Example

[3] https://doi.org/10.2210/pdb9R1O/pdb

PDB	`path("9R1O.pdb")`	RCSB PDB entry 9R1O.
mmCIF	`path("1FYY.cif")`	RCSB PDB entry 1FYY.
BinaryCIF	`path("1CRN.bcif")`	RCSB PDB entry 1CRN.
XYZ	`path("molecule.xyz")`	A standard atom-count/comment/coordinates XYZ file.

For documents that must also compile on Typst 0.14, read the file in the caller document and pass the resulting bytes. Always use `encoding: none` for external structure files; it preserves BinaryCIF bytes and avoids Unicode decoding loss for fixed-width PDB columns.

For the same files, use `read("9R1O.pdb", encoding: none)`, `read("1FYY.cif", encoding: none)`, or `read("1CRN.bcif", encoding: none)`. Complete, rendered examples for all three archive formats appear below.

> These files are RCSB PDB / wwPDB entries 9R1O, 1FYY, and 1CRN. Their PDB DOIs are 9R1O[4], 1FYY[5], and 1CRN[6]. PDB archive data files are available under CC0 1.0.

Passing a Typst string is accepted for small inline examples. A string is treated as inline molecular text, not as a file path.

II.2 XYZ Example

The following real-data example renders ethanol from the first PubChem3D conformer for PubChem CID 702. Save these two files with the shown `data/ethanol.xyz` relative path, then compile `ethanol-xyz.typ`.

PubChem ethanol conformer in XYZ format — *data/ethanol.xyz*

```
9
PubChem CID 702 (ethanol), PubChem3D conformer 000002BE00000001
O  -1.1712   0.2997   0.0000
C  -0.0463  -0.5665   0.0000
C   1.2175   0.2668   0.0000
H  -0.0958  -1.2120   0.8819
H  -0.0952  -1.1938  -0.8946
H   2.1050  -0.3720  -0.0177
H   1.2426   0.9307  -0.8704
H   1.2616   0.9052   0.8886
H  -1.1291   0.8364   0.8099
```

[4] https://doi.org/10.2210/pdb9R1O/pdb
[5] https://doi.org/10.2210/pdb1FYY/pdb
[6] https://doi.org/10.2210/pdb1CRN/pdb

Render PubChem ethanol from XYZ — *ethanol-xyz.typ*

```
#import "@preview/molfig:0.1.4"

#set page(width: 82mm, height: 82mm, margin: 4mm)

// PubChem CID 702 (ethanol), PubChem3D conformer 000002BE00000001.
// Record: https://pubchem.ncbi.nlm.nih.gov/compound/702
// Retrieved through PubChem PUG REST on 2026-08-24.
#let xyz = path("data/ethanol.xyz")

#molfig.render(
  xyz,
  format: "xyz",
  representation: "default",
  color-theme: "element-symbol",
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "svg",
  config: (
    azimuth: 125,
    elevation: 40,
    background: "",
  ),
  width: 74mm,
  height: 74mm,
)
```

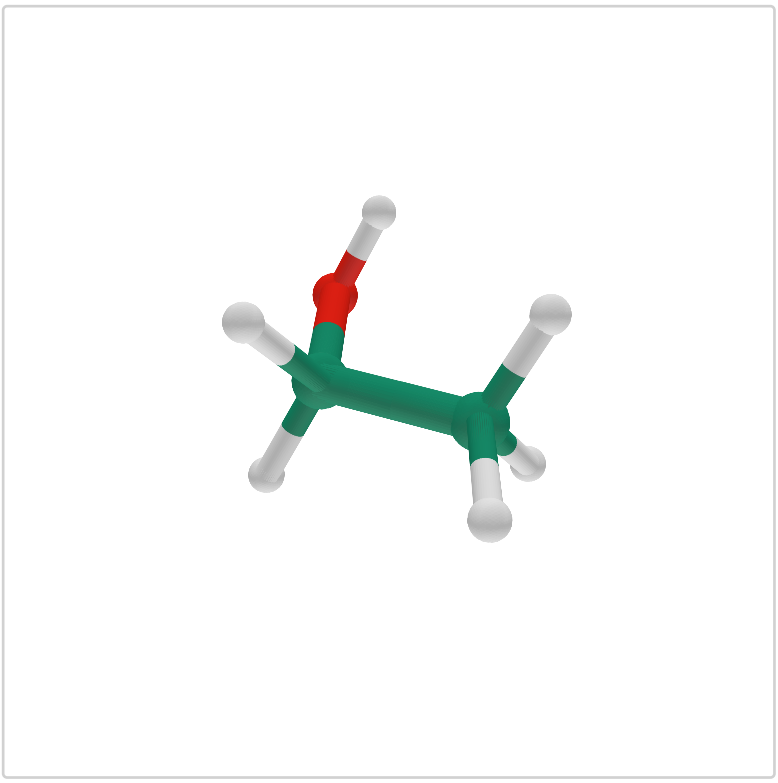

Figure 2: PubChem CID 702 (ethanol) rendered from XYZ with the complete settings above.

> Coordinate source: PubChem CID 702[7], PubChem3D conformer `000002BE00000001`. The 3D SDF was retrieved through PubChem PUG REST[8] on 2026-08-24 and transcribed to XYZ without changing atom order or coordinates. General PubChem citation: Kim et al. (2025), doi:10.1093/nar/gkae1059[9].

II.2.1 XYZ With Illustrative Style

`style: "illustrative"` can be applied to an XYZ structure independently of its representation and color theme. For this small XYZ structure, `representation: "default"` resolves to the Viewer ball-and-stick preset. The example keeps that representation and `color-theme: "element-symbol"` fixed so that only the style changes.

[7] https://pubchem.ncbi.nlm.nih.gov/compound/702
[8] https://pubchem.ncbi.nlm.nih.gov/docs/pug-rest
[9] https://doi.org/10.1093/nar/gkae1059

Compare default and illustrative styles for XYZ — *xyz-illustrative.typ*

```
// PubChem CID 241 (benzene), PubChem3D conformer 000000F100000001.
// Record: https://pubchem.ncbi.nlm.nih.gov/compound/241
// 3D SDF retrieved through PubChem PUG REST on 2026-08-24, then
// transcribed to XYZ without changing atom order or coordinates.
#import "@preview/molfig:0.1.4"
#set page(width: 124mm, height: auto, margin: 4mm)
#set text(font: "New Computer Modern", size: 9pt)
#let xyz = path("data/benzene.xyz")
#let view(style) = molfig.render(
  xyz,
  format: "xyz",
  representation: "default",
  color-theme: "element-symbol",
  style: style,
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "png",
  config: (azimuth: 35, elevation: 80, background: "", antialias: 4),
  width: 55mm,
  height: 50mm,
)
#let panel(label, style) = [
  #align(center, strong(label))
  #block(
    width: 55mm,
    height: 50mm,
    clip: true,
    align(center + horizon, view(style)),
  )
]
#grid(
  columns: (1fr, 1fr),
  column-gutter: 4mm,
  panel([Default], "default"),
  panel([Illustrative], "illustrative"),
)
```

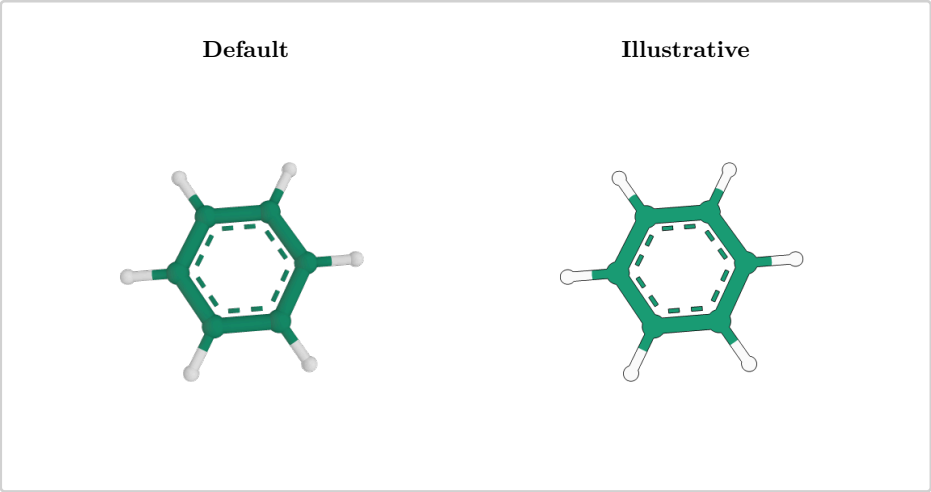

Figure 3: The same benzene XYZ ball-and-stick representation with default and Illustrative renderer styles.

II.2.2 XYZ Representation Comparison

XYZ supplies atoms and Cartesian coordinates but no polymer hierarchy, residues, or secondary-structure annotations. Consequently, its most useful distinct representations are `ball-and-stick`, `spacefill`, and `surface`. The comparison below fixes the source data, default style, camera, quality, and output settings. It uses the same benzene XYZ data as the Mol* comparison fixture. For this XYZ structure, `representation: "default"` resolves to ball-and-stick, so a duplicate default panel is omitted.

Compare XYZ representations — *xyz-representations.typ*

```
// PubChem CID 241 (benzene), PubChem3D conformer 000000F100000001.
// Record: https://pubchem.ncbi.nlm.nih.gov/compound/241
// 3D SDF retrieved through PubChem PUG REST on 2026-08-24, then
// transcribed to XYZ without changing atom order or coordinates.
#import "@preview/molfig:0.1.4"
#set page(width: 180mm, height: auto, margin: 4mm)
#set text(font: "New Computer Modern", size: 9pt)
#let xyz = path("data/benzene.xyz")
#let view(representation) = molfig.render(
  xyz,
  format: "xyz",
  representation: representation,
  style: "default",
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "png",
  config: (azimuth: 35, elevation: 80, background: "", antialias: 4),
  width: 54mm,
  height: 50mm,
)
#let panel(label, representation) = [
  #align(center, strong(label))
  #block(
    width: 54mm,
    height: 50mm,
    clip: true,
    align(center + horizon, view(representation)),
  )
]
#grid(
  columns: (1fr, 1fr, 1fr),
  column-gutter: 5mm,
  // Mol* Viewer Default resolves small XYZ structures to Ball & Stick.
  panel([Ball-and-stick], "default"),
  panel([Spacefill], "spacefill"),
  panel([Surface], "surface"),
)
```

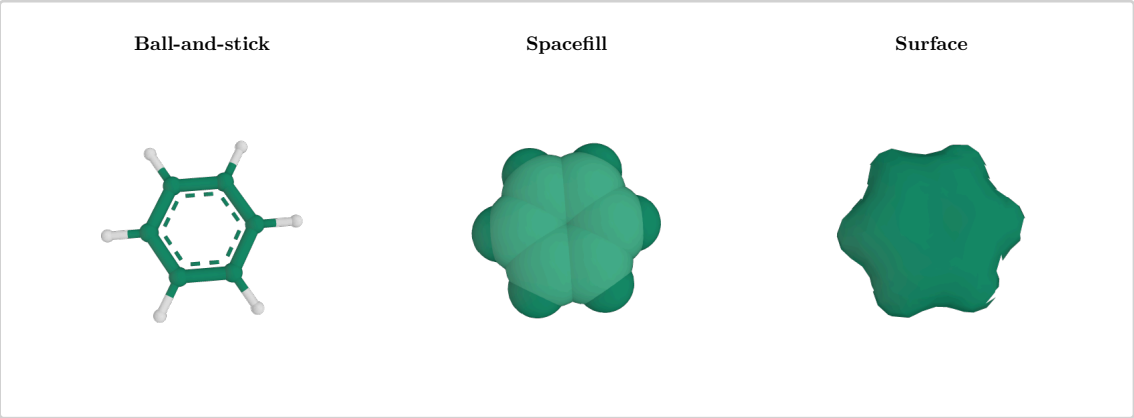

Figure 4: Ball-and-stick, spacefill, and molecular surface representations of the same benzene XYZ data.

The Surface panel exposes a known maquette 0.1.3 rendering limitation. Molfig exports the molecular-surface geometry together with indexed OBJ corner normals, but maquette reduces the normal indices of each face to one triangle normal before smooth shading. Its rendered result can therefore contain triangular fringes, faceted patches, or shading that differs from Mol*, even when the exported geometry is correct. This is not specific to XYZ: the same limitation applies to Surface meshes originating from PDB, mmCIF, and BinaryCIF. For a strict visual comparison, export OBJ and use a renderer that preserves indexed corner normals.

Coordinate sources: the style example uses PubChem CID 241[10] (benzene), conformer `000000F100000001`. Both comparisons use this same XYZ record. The 3D SDF was retrieved through PubChem PUG REST[11] on 2026-08-24 and transcribed to XYZ without changing atom order or coordinates. General PubChem citation: Kim et al. (2025), doi:10.1093/nar/gkae1059[12].

[10] https://pubchem.ncbi.nlm.nih.gov/compound/241
[11] https://pubchem.ncbi.nlm.nih.gov/docs/pug-rest
[12] https://doi.org/10.1093/nar/gkae1059

II.3 Choosing The First Options

Question	Recommended setting	Why
I want the Mol* Viewer Cartoon style.	`representation: "cartoon"`	Uses the Viewer Quick Styles Cartoon preset: polymer cartoon plus atomic detail for ligands and other non-polymer components.
I want only the polymer cartoon.	`representation: "polymer-cartoon"`	Uses the Mol* Cartoon provider defaults: polymer trace, nucleotide ring, and polymer gap visuals.
I need readable geometry diffs.	`mesh-format: "obj"`	OBJ is text and preserves face groups.
I need compact triangle bytes.	`mesh-format: "stl"`	STL is binary and simple for downstream mesh tools.
I want text mesh plus group metadata.	`mesh-format: "ply"`	PLY is ASCII and carries package-owned group comments/properties.
Compilation is slow.	`quality: "auto"` or `decimate: 0.3`	Lets Molfig pick lower tessellation for large structures or apply semantic molecular level-of-detail.

Part III

Public API

Every public command takes `(data)` as its first positional argument. It can be bytes from `read(..., encoding: none)`, inline molecular text as a string, or a Typst 0.15+ `path(...)` value. The same mesh options are shared by render, render-object, export, and metadata commands unless noted otherwise.

Command	Returns	Use when
`#render`	content	You want the figure directly in the document.
`#render-object`	dictionary	You want the rendered content plus raw mesh bytes and metadata.
`#to-obj`	bytes	You need OBJ mesh bytes for maquette or an external tool.
`#to-mtl`	bytes	You need the companion material library for OBJ output.
`#to-stl`	bytes	You need binary STL triangle data.
`#to-ply`	bytes	You need ASCII PLY output with Molfig metadata comments/properties.
`#info`	dictionary	You want structure and render planning metadata without rendering.
`#mesh-info`	dictionary	You want maquette's mesh metadata for the generated OBJ/STL/PLY bytes.

III.1 Rendering Commands

```
#render(
  (data),
  (format),
  (mesh-format),
  (representation),
  (color-theme),
  (style),
  (style-params),
  (config),
  (width),
  (height),
  (output-format)
)
```

Converts molecular bytes to a static mesh and delegates rendering to maquette.

Argument

`(data)` `bytes` | `str`

Molecular structure input. Pass `path("file.pdb")` on Typst 0.15+, or `read("file.pdb", encoding: none)` for Typst 0.14-compatible documents.

Argument

`(format): "auto"` `str`

Input parser selection. Use an explicit format for reproducible documents.

Argument

`(mesh-format): "obj"` `str`

Intermediate mesh format sent to maquette.

Argument

`(config): (:)` `dictionary`

JSON-encoded and passed to maquette. For OBJ rendering, Molfig adds the active color theme as a maquette material map; explicit `config.materials` entries override generated colors with the same material id.

Argument

`(width): "auto"` `str`

Width forwarded to the maquette rendering command.

Argument

`(height): "auto"` `str`

Height forwarded to the maquette rendering command.

Argument

`(output-format): "png"` `str`

Output format forwarded to maquette. PNG uses maquette's Z-buffer raster output and is recommended for high-poly meshes, large assemblies, and spacefill representations. SVG preserves vector geometry, but is intended for small to moderately sized meshes because each visible mesh face contributes SVG content that Typst must parse.

`#render-object((data), (mesh-format))`

Returns a dictionary with:

Key	Value
`kind`	The string `"render-object"`.
`format`	The selected mesh format.
`mesh_format`	The selected mesh format with snake_case naming.

`mesh`	Generated OBJ/STL/PLY bytes.
`materials`	Generated maquette material map for OBJ, or an empty dictionary for STL/PLY.
`info`	The same dictionary returned by `#info` for the same mesh options.
`content`	The Typst content returned by `#render`.

Render and inspect RCSB PDB entry 1FYY — *render-object.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 1FYY.
// https://doi.org/10.2210/pdb1FYY/pdb (CC0 1.0)
#let object = molfig.render-object(
  read("1FYY.cif", encoding: none),
  format: "cif",
  representation: "cartoon",
  mesh-format: "obj",
  assembly: "1",
  config: (azimuth: 30, elevation: 18, background: ""),
  width: 92mm,
  height: 64mm,
  output-format: "svg",
)

#object.content
#align(center)[
  Atoms: #object.info.atom_count \
  Mesh format: #object.mesh_format
]
```

Atoms: 797
Mesh format: obj

Figure 5: The complete `render-object` example for 1FYY, including values read from its returned metadata.

Structural data source: RCSB PDB / wwPDB, PDB ID `1FYY`, https://doi.org/10.2210/pdb1FYY/pdb[13]. PDB archive data files are available under CC0 1.0. Primary citation: Volk et al., *Biochemistry* 39, 14040–14053 (2000), doi:10.1021/bi001669l[14].

III.2 Export Commands

`#to-obj((data))`

Returns OBJ bytes. OBJ is the most useful interchange format when a downstream tool wants text geometry, material references, and group labels.

`#to-mtl((data))`

Returns the companion MTL material library for OBJ output. Use it alongside `#to-obj` when exporting outside Typst.

`#to-stl((data))`

Returns binary STL bytes. STL follows Mol* static exporter behavior: the header starts with `"Exported from Mol*"` and the two-byte facet attribute field is kept at zero.

`#to-ply((data))`

Returns ASCII PLY bytes. PLY is a Molfig-owned static mesh format because the pinned Mol* geo-export extension does not provide PLY output.

[13] https://doi.org/10.2210/pdb1FYY/pdb
[14] https://doi.org/10.1021/bi001669l

Export RCSB PDB entry 1CRN to every mesh format — *exports.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 1CRN.
// https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
#let data = read("1CRN.bcif", encoding: none)
#let obj = molfig.to-obj(data, format: "bcif", representation: "cartoon")
#let mtl = molfig.to-mtl(data, format: "bcif", representation: "cartoon")
#let stl = molfig.to-stl(data, format: "bcif", representation: "cartoon")
#let ply = molfig.to-ply(data, format: "bcif", representation: "cartoon")

#table(
  columns: (1fr, 1fr),
  table.header([*Export*], [*Generated bytes*]),
  [OBJ], [#obj.len()],
  [MTL], [#mtl.len()],
  [STL], [#stl.len()],
  [PLY], [#ply.len()],
)
```

Export	Generated bytes
OBJ	602244
MTL	92
STL	1542684
PLY	358830

Figure 6: Byte lengths produced by the complete 1CRN export example.

Structural data source: RCSB PDB / wwPDB, PDB ID `1CRN`, https://doi.org/10.2210/pdb1CRN/pdb[15]. PDB archive data files are available under CC0 1.0. Primary citation: Teeter, *Proceedings of the National Academy of Sciences* 81, 6014–6018 (1984), doi:10.1073/pnas.81.19.6014[16].

[15] https://doi.org/10.2210/pdb1CRN/pdb
[16] https://doi.org/10.1073/pnas.81.19.6014

III.3 Metadata Commands

#info(⟨data⟩)

Parses the structure and returns molecular and mesh-planning metadata without maquette rendering. This is the fastest public command for debugging parser, assembly, altLoc, bond, secondary structure, and representation behavior.

#mesh-info(⟨data⟩, ⟨mesh-format⟩)

Generates a mesh, then delegates mesh metadata extraction to maquette's `get-obj-info`, `get-stl-info`, or `get-ply-info` helper.

Part IV

Shared Mesh Options

The following options are accepted by `#render`, `#render-object`, `#to-obj`, `#to-mtl`, `#to-stl`, `#to-ply`, and `#info`.

IV.1 Input Selection

Option	Default	Meaning
`⟨format⟩`	`"auto"`	Input parser. Supported values: `"auto"`, `"pdb"`, `"cif"`, `"mmcif"`, `"bcif"`, `"binarycif"`, `"binary-cif"`, `"xyz"`.
`⟨block-index⟩`	`none`	Zero-based BinaryCIF data block index. Useful when one BinaryCIF file contains multiple data blocks.
`⟨block-header⟩`	`""`	Exact BinaryCIF block header. When set, it takes precedence over `⟨block-index⟩`.

Format	Typical extension	Notes
`"pdb"`	`.pdb`	Fixed-width text. Keep bytes to preserve columns.
`"cif"` or `"mmcif"`	`.cif`, `.mmcif`	Text CIF categories and loops.
`"bcif"`	`.bcif`	BinaryCIF MessagePack data with encoded columns and optional masks.
`"xyz"`	`.xyz`	Atom count, comment line, and element/Cartesian-coordinate records. Multiple frames become models; rendering selects the first model, matching Mol* defaults.
`"auto"`	any	Convenient for exploration, but explicit formats are better for release documents.

BinaryCIF numeric atom-site columns are decoded into typed accessors before falling back to string cells. Molfig does not round-trip BinaryCIF through synthetic CIF text.

IV.2 Structure Selection

Option	Default	Meaning

⟨assembly⟩	"1"	Biological assembly id. Use "asymmetric-unit", "none", or an empty string for source coordinates without assembly operators.
⟨alt-loc⟩	""	Alternate location id or policy. The empty value follows Mol* and includes all locations; concrete ids such as "A", "highest-occupancy", and "all" are also supported.
⟨infer-bonds⟩	true	Infer covalent bonds when explicit bond data are absent.
⟨center⟩	true	Recenters exported vertices around Mol*-style visible export bounds.

Assembly expansion keeps the source model and unit operators as the semantic structure boundary, then materializes mesh coordinates for export. This means metadata can still report selected assembly operators even though OBJ/STL/PLY are static mesh formats.

IV.3 Representation Selection

Option	Default	Meaning
⟨representation⟩	"default"	Uses the Mol* Viewer configured default preset, falling back to its automatic size-dependent selection. Use "cartoon" to pin Viewer Quick Styles Cartoon or "polymer-cartoon" for the standalone Cartoon provider.
⟨color-theme⟩	"chain-id"	Selects "chain-id", "element-symbol", "entity-id", "operator-name", "plddt-confidence", "qmean-score", or "sb-ncbr-partial-charges". Color is serialized through OBJ material references and the companion MTL output. STL and the current PLY output do not carry these colors, so themed document rendering requires mesh-format: "obj".
⟨style⟩	"default"	Selects "default" or "illustrative". Style is independent of ⟨representation⟩ and ⟨color-theme⟩, so the same flat-light, outline, and occlusion treatment can be applied to Cartoon, Spacefill, Ball-and-stick, Surface, Ribbon, or Backbone geometry without changing their material colors.

⟨style-params⟩	(:)	Configures Illustrative rendering with ignore-light, outline, and occlusion all three default to true.
⟨theme⟩	(:)	Applies Mol* Viewer preset overrides. Supported keys are globalName, carbonColor, and symmetryColor. An empty dictionary preserves the preset's normal component-specific themes.

The representation names follow Mol* Viewer Quick Styles rather than only the low-level representation registry. In the Viewer, Cartoon applies the polymer-and-ligand preset; it is not limited to the standalone Cartoon provider. Spacefill is available as "spacefill". Surface corresponds to Mol*'s molecular-surface preset and is selected with representation: "surface". It uses the all component, entity-id colors with the water override, a physical size theme, probeRadius: 1.4, probePositions: 36, and the Mol* CPU marching-cubes path.

> "default" and "auto" preserve their distinct public names and follow Mol* size routing for Small, Medium, and Large structures. "default" first applies the Viewer annotation priority described below, while "auto" requests size routing directly. Huge and Gigantic structures use Mol*'s CPU Gaussian-surface branches, including trace-only and structure-level routing where prescribed by ViewerAuto.

IV.3.1 Cartoon, Spacefill, And Surface

The same structure and camera make the geometry difference explicit. Cartoon emphasizes polymer topology and secondary structure; Spacefill exposes atomic packing with van der Waals spheres; Surface shows the continuous solvent-excluded molecular envelope computed by the CPU molecular-surface path.

Compare three representations of RCSB PDB entry 1CRN *representations.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 1CRN.
// https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
#let data = read("1CRN.bcif", encoding: none)
#let view(rep) = molfig.render(
  data, format: "bcif", representation: rep,
  mesh-format: "obj", quality: "high", center: true,
  output-format: "svg",
  config: (azimuth: 35, elevation: 24, background: ""),
  width: 55mm, height: 50mm,
)
#let panel(label, rep) = [
  #align(center, strong(label))
  #block(
    width: 55mm, height: 50mm, clip: true,
    align(center + horizon, scale(
      x: 165%, y: 165%, origin: center + horizon, view(rep),
    )),
  )
]
#grid(
  columns: (1fr, 1fr, 1fr), column-gutter: 2mm,
  panel([Cartoon], "cartoon"),
  panel([Spacefill], "spacefill"),
  panel([Surface], "surface"),
)
```

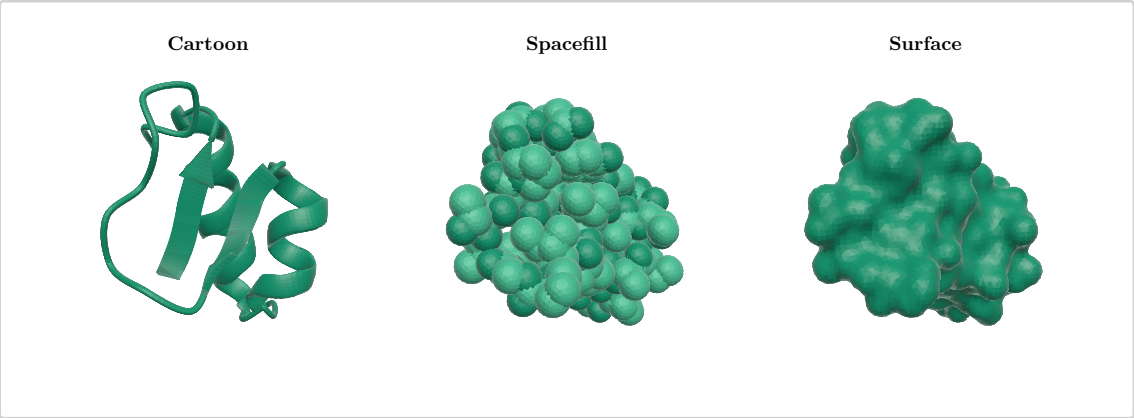

Figure 7: RCSB PDB entry 1CRN rendered with identical camera and quality settings as Cartoon, Spacefill, and Surface.

Structural data source: RCSB PDB / wwPDB, PDB ID `1CRN`, https://doi.org/10.2210/pdb1CRN/pdb[17]. PDB archive data files are available under CC0 1.0. Primary citation: Teeter, *Proceedings of the National Academy of Sciences* 81, 6014–6018 (1984), doi:10.1073/pnas.81.19.6014[18].

IV.3.2 Illustrative Style

`⟨style⟩` is a renderer treatment, not a representation or color theme. Molfig first constructs the selected `⟨representation⟩` and resolves the selected `⟨color-theme⟩`. With `style: "illustrative"`, it preserves that geometry and those material colors, then asks maquette to approximate the Mol* Quick Styles neutral unlit colors, dark outlines, ambient occlusion, and disabled shadows.

Illustrative rendering is not pixel-identical to the Mol* WebGL renderer. Molfig reproduces Mol*-derived static geometry and material selection, but OBJ, STL, and PLY do not carry Mol*‘s camera-dependent outline, screen-space ambient occlusion, depth reconstruction, blur, or final color composition. Maquette computes those effects with its own rasterizer and post-processing algorithms. Simple XYZ atom-and-bond figures can therefore look very close to Mol*, while folded polymer Cartoon, Ribbon, and Backbone geometry can show more visible differences in internal contours and occlusion shading. The limitation follows the realized geometry; PDB, mmCIF, and BinaryCIF representations of the same structure are subject to the same renderer difference.

[17] https://doi.org/10.2210/pdb1CRN/pdb
[18] https://doi.org/10.1073/pnas.81.19.6014

Apply Illustrative style without changing representation or color theme *illustrative.typ*

```
// Structural data: RCSB PDB / wwPDB entry 9Z4O (CC0 1.0).
// https://doi.org/10.2210/pdb9Z4O/pdb
#import "@preview/molfig:0.1.4"
#set page(width: 124mm, height: auto, margin: 4mm)
#set text(font: "New Computer Modern", size: 9pt)
#let data = path("data/9Z4O.pdb")
#let view(style) = molfig.render(
  data,
  format: "pdb",
  representation: "cartoon",
  assembly: "1",
  color-theme: "chain-id",
  style: style,
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "png",
  config: (up: (0, 1, 0), azimuth: 35, elevation: 24, background: "", antialias: 4),
  width: 55mm,
  height: 50mm,
)
#let panel(label, style) = [
  #align(center, strong(label))
  #block(
    width: 55mm,
    height: 50mm,
    clip: true,
    align(center + horizon, scale(
      x: 145%, y: 145%, origin: center + horizon, view(style),
    )),
  )
]
#grid(
  columns: (1fr, 1fr),
  column-gutter: 4mm,
  panel([Default], "default"),
  panel([Illustrative], "illustrative"),
)
```

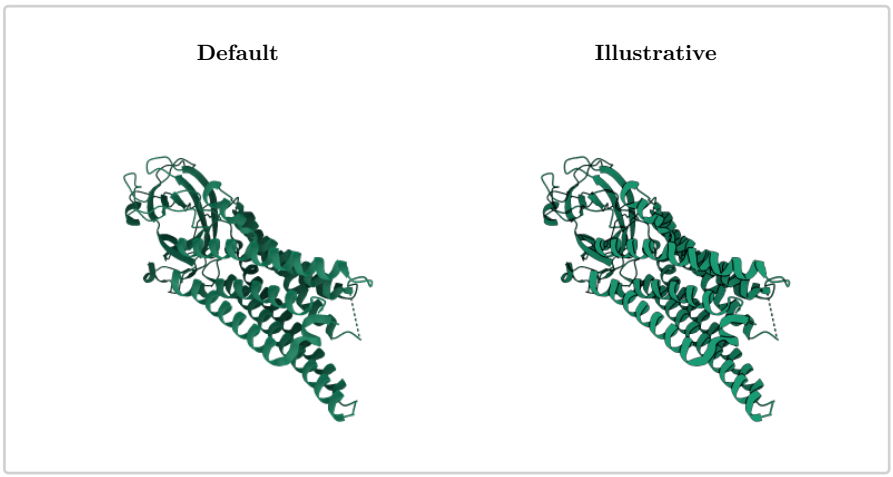

Figure 8: RCSB PDB entry 9Z4O rendered twice with identical Cartoon geometry and chain-id colors; only the renderer style changes.

Structural data source: RCSB PDB / wwPDB, PDB ID `9Z4O`, https://doi.org/10.2210/pdb9Z4O/pdb[19]. PDB archive data files are available under CC0 1.0. Structural data from the wwPDB archive.

For `#render` and `#render-object`, `ignore-light: true` uses maquette's `shading: "flat"` path, fixes both hemispheric ambient poles to white at neutral intensity, supplies a zero-intensity directional light, and disables specular and Fresnel terms. This preserves the Mol* material's sRGB base color instead of tinting it through maquette's default environment. `outline: true` requests an outline for every representation. Molfig maps the Mol* Quick Style intent to maquette's AA-quantized depth-edge detector: polymer geometry uses width 0.9; small atomistic and surface figures use a black width-1.5 contour; dense atomistic figures use a near-black width-1.0 contour to avoid multiplying internal mesh edges into a black cast. The choice is derived from realized geometry and atom-count metadata, not from the input format. This renderer adaptation keeps the style API independent of the selected representation, but it does not reproduce Mol*'s outline shader. `occlusion: true` enables maquette SSAO, and shadows are disabled. Mol* and maquette use different SSAO radius units and kernels, so Molfig maps the same Quick Style to a broad polymer kernel and a local atomistic/surface kernel instead of copying the numeric radius. The resulting shading is an approximation because the two renderers reconstruct depth, sample occlusion, blur, and compose color differently. User-supplied `(config)` keys take precedence over all of these defaults. SSAO is available for PNG rendering; SVG retains the flat lighting and silhouette outline. Because these are renderer effects, OBJ and MTL output is identical for default and Illustrative styles.

IV.3.3 Viewer Theme Overrides

Mol* Viewer presets accept a `theme` dictionary through their common representation parameters. `globalName` replaces the provider theme for each component, `carbonColor` controls carbon atoms in ball-and-stick ligand, non-standard, and branched components, and `symmetryColor` replaces

[19] https://doi.org/10.2210/pdb9Z4O/pdb

the polymer theme only for non-assembly crystallographic symmetry units. Water, ion, and lipid components keep element-symbol carbon coloring as in the Viewer preset.

Apply Mol* Viewer theme overrides to RCSB PDB entry 1CRN *theme.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 1CRN.
// https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
#molfig.render(
  read("1CRN.bcif", encoding: none),
  format: "bcif",
  representation: "cartoon",
  theme: (
    globalName: "element-symbol",
    carbonColor: "chain-id",
    symmetryColor: "operator-name",
  ),
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "svg",
  config: (azimuth: 35, elevation: 24, background: ""),
  width: 92mm,
  height: 68mm,
)
```

Figure 9: RCSB PDB entry 1CRN with the Viewer theme override example applied.

Structural data source: RCSB PDB / wwPDB, PDB ID `1CRN`, https://doi.org/10.2210/pdb1CRN/pdb [20]. PDB archive data files are available under CC0 1.0. Primary citation: Teeter (1984), doi:10.1073/pnas.81.19.6014[21].

IV.3.4 Annotation Color Rules

With `representation: "default"`, Molfig follows the ViewerAuto preset and selects the first applicable embedded annotation in this order: `plddt-confidence`, `qmean-score`, then `sb-ncbr-partial-charges`. These names may also be selected explicitly with `(color-theme)`. `representation: "auto"` skips annotation selection and performs only structure-size routing.

Theme	Value rule	Colors
`plddt-confidence`	`score <= 50`, `<= 70`, `<= 90`, then `> 90`	`#ff7d45` , `#ffdb13` , `#65cbf3` , `#0053d6`
`qmean-score`	Values through `0.5` use orange; `0.5..1.0` interpolates to blue.	`#ff5000` to `#025afd`
`sb-ncbr-partial-charges`	Residue charge: negative through zero to positive.	`#ff0000` through `#ffffff` to `#0000ff`

[20] https://doi.org/10.2210/pdb1CRN/pdb
[21] https://doi.org/10.1073/pnas.81.19.6014

Missing pLDDT values fall back to `B_iso_or_equiv`. Unavailable or negative quality scores use `#aaaaaa` . Missing partial-charge locations use `#66ff00` , matching the Mol* provider. Annotation colors require OBJ when rendered through maquette because STL and the current PLY schema have no material color channel.

IV.3.5 Chain ID Color Rules

The `"chain-id"` theme follows these rules:

- For atomic models, the author chain id (`auth_asym_id`) is used when it is present; otherwise the label chain id (`label_asym_id`) is used. PDB input normally has the same value for both. Coarse IHM spheres and Gaussians use their asym id.
- Unique chain ids are collected in model order. Atomic chains are collected first, followed by previously unseen coarse-sphere and coarse-Gaussian chain ids. Assembly copies retain the source chain id and therefore retain its color.
- Colors are assigned from the following 25-color Mol* many-distinct palette. If a structure contains more than 25 chain ids, the palette repeats from the first color.

Palette positions	Colors
1–5	`#1b9e77` , `#d95f02` , `#7570b3` , `#e7298a` , `#66a61e`
6–10	`#e6ab02` , `#a6761d` , `#666666` , `#e41a1c` , `#377eb8`
11–15	`#4daf4a` , `#984ea3` , `#ff7f00` , `#ffff33` , `#a65628`
16–20	`#f781bf` , `#999999` , `#66c2a5` , `#fc8d62` , `#8da0cb`
21–25	`#e78ac3` , `#a6d854` , `#ffd92f` , `#e5c494` , `#b3b3b3`

Chain-associated geometry, including cartoon, ribbon, backbone, nucleotide, carbohydrate, gap, and coarse visuals, uses the assigned chain color unless the visual has an explicit atomic material. Atomic visuals apply a chain-and-element rule:

- Carbon uses its chain color for ordinary atoms, polymers, ligands, and branched components.
- Water, ion, and lipid components use element-symbol colors for every atom, including carbon.
- Non-carbon atoms use their element-symbol color. The symbol is read from `type_symbol` when available, otherwise from the parsed element field. Unknown symbols fall back to white.
- Bond geometry inherits the material of its source atom. Component visuals that emit two directed half-bonds consequently color each half from its adjacent atom.

Element-symbol colors include Mol*‘s default lightness adjustment. The final RGB values written to OBJ materials are:

Elements	Final colors
Hydrogen isotopes	H `#ffffff` , D `#ffffca` , T `#ffffaa`
Common organic elements	C `#999999` , N `#4259ff` , O `#ff2618` , P `#ff8a14` , S `#ffff3e` , Se `#ffab17`
Halogens	F `#9aea5a` , Cl `#37fb2e` , Br `#b13431` , I `#9e179e`

Alkali and alkaline-earth metals	Na `#b566fd` , Mg `#95ff1f` , K `#994ade` , Ca `#4eff1e`
Transition metals	Mn `#a683d1` , Fe `#eb703c` , Co `#fb9aaa` , Ni `#5cda5a` , Cu `#d3893c` , Zn `#8689ba`
Other or unknown	`#ffffff`

The companion MTL records opacity `0.3` for branched components, `0.6` for water and lipids, and `1.0` otherwise. Molfig's automatic maquette material map currently forwards RGB colors only; it does not forward these MTL opacity values.

Value	Best for	Main geometry	Notes
`"default"`	Viewer-compatible automatic figures	ViewerAuto annotation theme, then size-selected geometry	Uses pLDDT, QMEAN, or SB-NCBR partial charges when applicable, with Gaussian surfaces for Huge/Gigantic structures.
`"auto"`	Explicit automatic selection	Atomic detail, polymer Cartoon, or a Gaussian surface according to structure size	Huge/Gigantic routing follows the ViewerAuto size thresholds.
`"cartoon"`	General figures	Polymer Cartoon plus atomic detail for non-polymers and carbohydrate visuals	Pins the Mol* Viewer Quick Styles Cartoon preset.
`"polymer-cartoon"`	Polymer-only figures	Polymer trace, nucleotide ring, and polymer gap visuals	Standalone Mol* Cartoon provider defaults.
`"spacefill"`	Atomic packing	Atom spheres	Matches Viewer Quick Styles Spacefill: entity-id base colors with carbon lightening when no theme override is supplied. `⟨style⟩` remains an independent renderer treatment.
`"surface"`	Solvent-accessible shape	CPU molecular-surface field and marching-cubes mesh	Viewer Quick Styles Surface with entity-id colors and red water override.
`"ball-and-stick"`	Ligands and small molecules	Atom spheres plus bond cylinders	Good for local chemistry and explicit bond inspection.
`"ribbon"`	Backbone shape	Ribbon-oriented polymer geometry	Good for broad fold visibility.

`"backbone"`	Trace-only views	Backbone cylinders and spheres	Lower-detail alternative for polymer paths.

IV.4 Geometry And Quality

Option	Default	Meaning
`⟨quality⟩`	`"custom"`	`"custom"` preserves explicit numeric controls. `"auto"` chooses a preset from structure size. Also accepts `"highest"`, `"higher"`, `"high"`, `"medium"`, `"low"`, `"lower"`, `"lowest"`.
`⟨decimate⟩`	`0`	Molfig-side molecular level-of-detail strength in `[0, 1]`. It reduces sphere detail, polymer curve/profile segments, surface resolution, probe sampling, and exported cylinder detail before maquette receives the mesh. In `#render` and `#render-object`, `config.decimate` is also consumed this way and is not forwarded to maquette's generic triangle decimator.
`⟨sphere-detail⟩`	`2`	Sphere tessellation detail. Public input is clamped by the plugin.
`⟨linear-segments⟩`	`8`	Curve subdivisions for polymer and tube paths.
`⟨radial-segments⟩`	`16`	Radial subdivisions for cylinders, tubes, and ribbon-like profiles.
`⟨radius-scale⟩`	`1.0`	Global radius multiplier.
`⟨atom-radius⟩`	`0.28`	Explicit atom radius for simple representations.
`⟨bond-radius⟩`	`0.12`	Explicit bond radius for cylinder-based bond visuals.
`⟨ribbon-radius⟩`	`0.2`	Tube/ribbon radius control.
`⟨ribbon-width⟩`	`0.55`	Ribbon width control.
`⟨helix-profile⟩`	`"elliptical"`	`"elliptical"`, `"rounded"`, or `"square"`. `"sheet"` is accepted as an alias for square.
`⟨round-cap⟩`	`false`	Enable rounded caps where supported by the selected cartoon/ribbon path.
`⟨sheet-arrow-factor⟩`	`1.5`	Scales sheet arrow geometry.
`⟨tubular-helices⟩`	`false`	Prefer tubular helix geometry in supported cartoon paths.

Quality presets override the explicit numeric tessellation controls. Use `quality: "custom"` when exact reproducibility matters, and use `quality: "auto"` or a lower preset when large structures compile too slowly. ⟨`decimate`⟩ is applied after the quality preset, so it can be used as an additional opt-in geometry budget without changing representation selection.

The sphere-detail column below describes the representation-side visual quality. For static sphere primitives, Molfig now also follows Mol* Geo Export: automatic export detail is 3 below 2,000 spheres, 2 below 20,000, and 1 otherwise. `quality: "custom"` continues to honor an explicit ⟨`sphere-detail`⟩.

Preset	Sphere detail	Radial segments	Linear segments
`"highest"`	3	36	18
`"higher"`	3	28	14
`"high"`	2	20	10
`"medium"`	1	12	8
`"low"`	0	8	3
`"lower"`	0	4	2
`"lowest"`	0	2	1

IV.4.1 Semantic Decimate

⟨`decimate`⟩ is intended for document builds where a molecularly meaningful lower-detail mesh is preferable to generic triangle clustering. The example below keeps the same PDB entry, quality preset, and camera, but compares practical semantic level-of-detail strengths for Cartoon, Surface, and Spacefill. Cartoon primarily reduces curve and profile segments, Surface raises the molecular-surface grid resolution and reduces probe sampling, and Spacefill reduces sphere tessellation before maquette receives the OBJ.

Compare Molfig semantic decimate strengths on RCSB PDB entry 1CRN *decimate.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 1CRN.
// https://doi.org/10.2210/pdb1CRN/pdb (CC0 1.0)
#let data = read("1CRN.bcif", encoding: none)
#let panel-width = 55mm
#let panel-height = 42mm

#let panel(name, representation, strength, zoom) = [
  #align(center)[#strong(name)\
    #text(size: 8pt)[decimate: #strength]]
  #block(width: panel-width, height: panel-height, clip: true)[
    #align(center + horizon, scale(
      x: zoom, y: zoom, origin: center + horizon,
      molfig.render(data, format: "bcif", representation:
      representation,
        mesh-format: "obj", quality: "high", center: true,
        output-format: "svg",
        config: (azimuth: 35, elevation: 24, background: "",
          decimate: strength),
        width: panel-width, height: panel-height),
    ))
  ]
]

#grid(
  columns: (1fr, 1fr, 1fr), column-gutter: 2mm, row-gutter: 3mm,
  panel([Cartoon], "cartoon", 0, 165%),
  panel([Cartoon], "cartoon", 0.35, 165%),
  panel([Cartoon], "cartoon", 0.65, 165%),
  panel([Surface], "surface", 0, 145%),
  panel([Surface], "surface", 0.35, 145%),
  panel([Surface], "surface", 0.65, 145%),
  panel([Spacefill], "spacefill", 0, 145%),
  panel([Spacefill], "spacefill", 0.35, 145%),
  panel([Spacefill], "spacefill", 0.65, 145%),
)
```

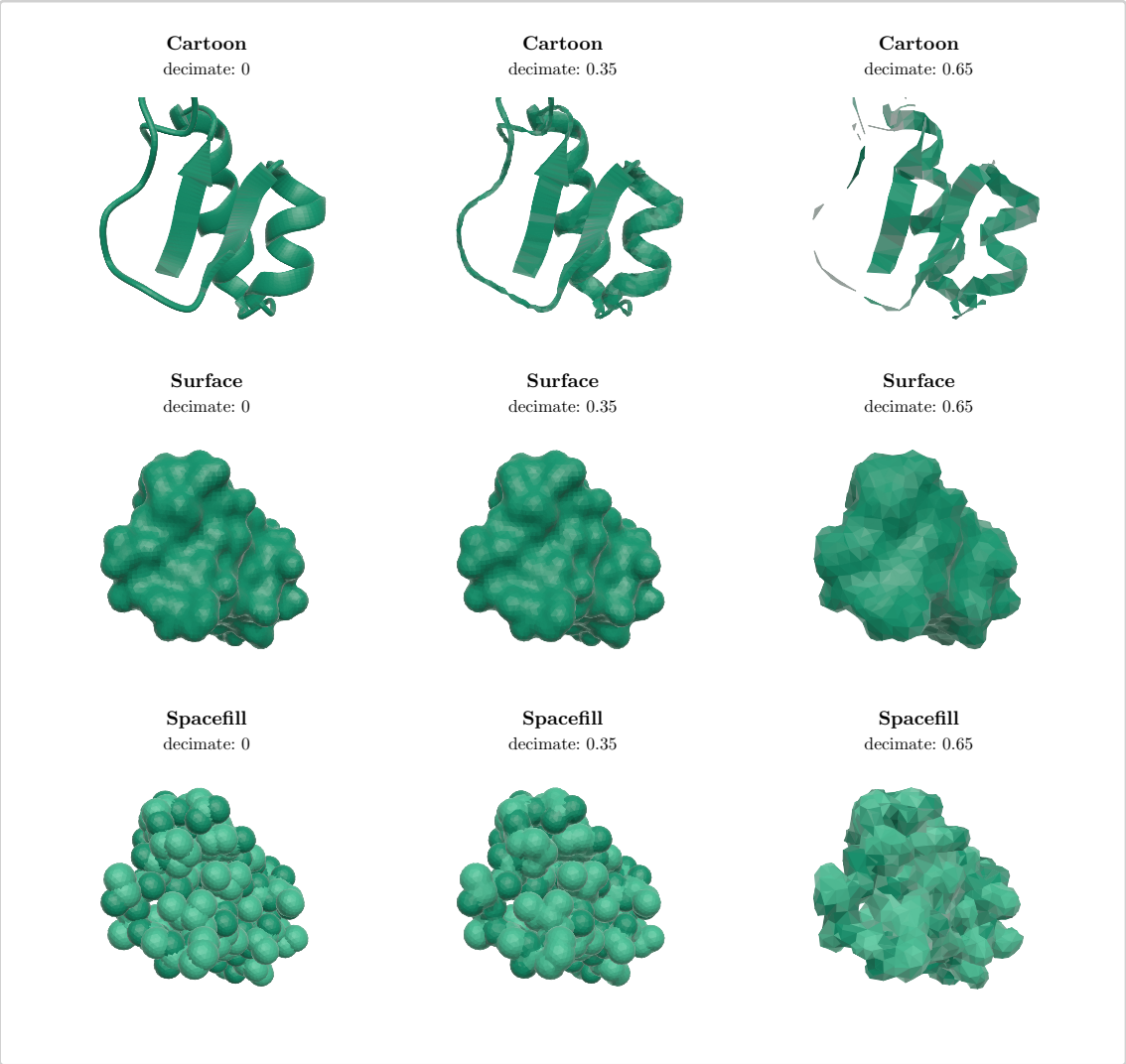

Figure 10: RCSB PDB entry 1CRN rendered as Cartoon, Surface, and Spacefill with practical Molfig semantic decimate strengths.

Structural data source: RCSB PDB / wwPDB, PDB ID `1CRN`, https://doi.org/10.2210/pdb1CRN/pdb[22]. PDB archive data files are available under CC0 1.0. Primary citation: Teeter, *Proceedings of the National Academy of Sciences* 81, 6014–6018 (1984), doi:10.1073/pnas.81.19.6014[23].

The public default is `quality: "custom"`, so the numeric defaults in this manual are honored unless you explicitly choose a preset or `"auto"`.

[22] https://doi.org/10.2210/pdb1CRN/pdb
[23] https://doi.org/10.1073/pnas.81.19.6014

Part V

Maquette Rendering

`#render` calls one of maquette's mesh renderers based on `(mesh-format)`: `render-obj`, `render-stl`, or `render-ply`. The `(config)` dictionary is JSON-encoded and passed through. For OBJ, Molfig derives maquette's `materials` dictionary from the exported material ids so `color-theme: "chain-id"` is visible in the document. User-supplied material entries are merged last and therefore take precedence. With Illustrative style, user-supplied maquette lighting, ambient, specular, Fresnel, outline, SSAO, and shadow keys take precedence over Molfig's defaults. STL has no material channel, and Molfig's current PLY schema contains no vertex or face color properties. Consequently, maquette renders STL and PLY without the selected color theme.

> In maquette 0.1.3, OBJ and PLY corner normals are not retained with their original per-corner indexing through smooth shading. Curved Surface meshes are particularly sensitive to this and may show shading artifacts that are absent in Mol*. The raw Molfig mesh and the in-document maquette rendering must therefore be evaluated separately when checking Surface parity.

The complete 1FYY `render-object` example above demonstrates camera, background, dimensions, and raster-output passthrough, and its rendered result shows the exact content returned in `object.content`.

Molfig does not validate maquette-specific keys. Unknown keys are passed to maquette, so maquette remains the source of truth for camera and renderer settings.

V.1 Choosing A Mesh Format

Format	Command	Strength	Tradeoff
OBJ	`#to-obj`	Readable text, group labels, material references	Larger than binary STL and needs MTL when material rows are consumed externally.
MTL	`#to-mtl`	Companion material rows for OBJ	Not a geometry format by itself.
STL	`#to-stl`	Compact binary triangle stream	No stable text metadata channel; facet attribute bytes are zero.
PLY	`#to-ply`	Text mesh with Molfig group metadata	Package-owned output, not a pinned Mol* exporter format.

For visual documents, start with OBJ. Switch to STL only when the downstream consumer specifically wants binary STL, and use PLY when the group metadata is more useful than OBJ/MTL compatibility.

Part VI

Metadata Reference

`#info` returns a dictionary. It is intended for document logic, regression tests, and debugging. The exact set of keys may grow as Molfig's Mol*-parity layer grows, but the current major groups are:

Key	Type	Meaning
`atom_count`	integer	Visible atom count after selected assembly/altLoc handling and geometry expansion.
`bond_count`	integer	Visible bond count used by static geometry.
`alt_locs`	array	Available alternate location ids from the source coordinates.
`alt_locs_info`	dictionary	Selected altLoc policy and available ids.
`assemblies`	array	Available biological assembly summaries.
`assembly`	dictionary	Selected assembly id.
`source_data`	dictionary	Input source categories, row counts, and original source kind.
`structure`	dictionary	Model/Structure/Unit counts, boundary, conformation, segments, ranges, and lookup3d summary.
`secondary_structure`	dictionary	Helix and sheet ranges.
`representation`	dictionary	Selected and realized visual names, plus the selected style and resolved Illustrative parameters.
`render_objects`	array	Semantic render-object spans with geometry type, visual, chain, residue range, group id, component, representation tag/order, provider color-theme metadata, and value-cell style counts.
`bond_metadata`	dictionary	Counts by bond source and flags such as struct_conn, index_pair, chem_comp, aromatic, and resonance.
`bounds`	dictionary	Expanded coordinate bounds before export centering.

Inspect the Model/Structure/Unit result for RCSB PDB entry 9R1O — *metadata.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 9R1O.
// https://doi.org/10.2210/pdb9R1O/pdb (CC0 1.0)
#let meta = molfig.info(
  read("9R1O.pdb", encoding: none),
  format: "pdb",
  representation: "cartoon",
  assembly: "1",
  alt-loc: "highest-occupancy",
)

#table(
  columns: (1fr, 1fr),
  table.header([*Property*], [*Value*]),
  [Atoms], [#meta.atom_count],
  [Bonds], [#meta.bond_count],
  [Assembly], [#meta.assembly.id],
  [Units], [#meta.structure.unit_count],
  [Realized visuals], [#meta.representation.realized_visuals.join(", ")],
)
```

Property	Value
Atoms	1503
Bonds	12
Assembly	1
Units	2
Realized visuals	polymer-trace, polymer-gap, element-sphere, intra-bond

Figure 11: Selected structure and representation metadata produced from PDB entry 9R1O.

Structural data source: RCSB PDB / wwPDB, PDB ID `9R1O`, https://doi.org/10.2210/pdb9R1O/pdb[24]. PDB archive data files are available under CC0 1.0. Deposition authors: Petrenas, Ozga, Chubb, and Woolfson.

[24] https://doi.org/10.2210/pdb9R1O/pdb

The `render_objects` array can be used for stronger assertions, for example checking that at least one object has `secondary_type == "helix"` or that `realized_visuals` contains `polymer-trace`.

VI.1 Assembly And Unit Metadata

Assembly selection is visible through both `info(...).structure` and mesh export metadata. OBJ and PLY include `molfig_operator_metadata` comments when an assembly is selected. STL has no equivalent text channel, so assembly operator metadata should be inspected through `#info` or `#render-object`.

Path	Example	Meaning
`structure.unit_count`	`meta.structure.unit_count`	Number of units after structure construction.
`structure.symmetry_group_count`	`meta.structure.symmetry_group_count`	Mol*-style symmetry grouping count.
`structure.coordinate_system`	`meta.structure.coordinate_system`	Selected coordinate operator summary.
`structure.boundary`	`meta.structure.boundary.sphere_radius`	Boundary sphere and box used by structure-level geometry.
`structure.lookup3d`	`meta.structure.lookup3d.unit_count`	Lookup3D summary for constructed units.

Part VII

Practical Workflows

VII.1 A Publication Figure

The complete 9R1O Quickstart is the publication-oriented example: it pins the PDB entry, biological assembly, representation, tessellation quality, camera, centering, mesh format, and raster output. Its code, rendered PDF, and source attribution are shown together in the Quickstart section.

VII.2 A Lightweight Draft Figure

Draft a large real assembly from RCSB PDB entry 9M1U — *9M1U.typ*

```
#import "@preview/molfig:0.1.4"

// Structural data: RCSB PDB / wwPDB entry 9M1U.
// https://doi.org/10.2210/pdb9M1U/pdb (CC0 1.0)
#molfig.render(
  read("9M1U.pdb", encoding: none),
  format: "pdb",
  representation: "cartoon",
  assembly: "1",
  quality: "auto",
  mesh-format: "obj",
  output-format: "png",
  config: (elevation: 45, background: ""),
)
```

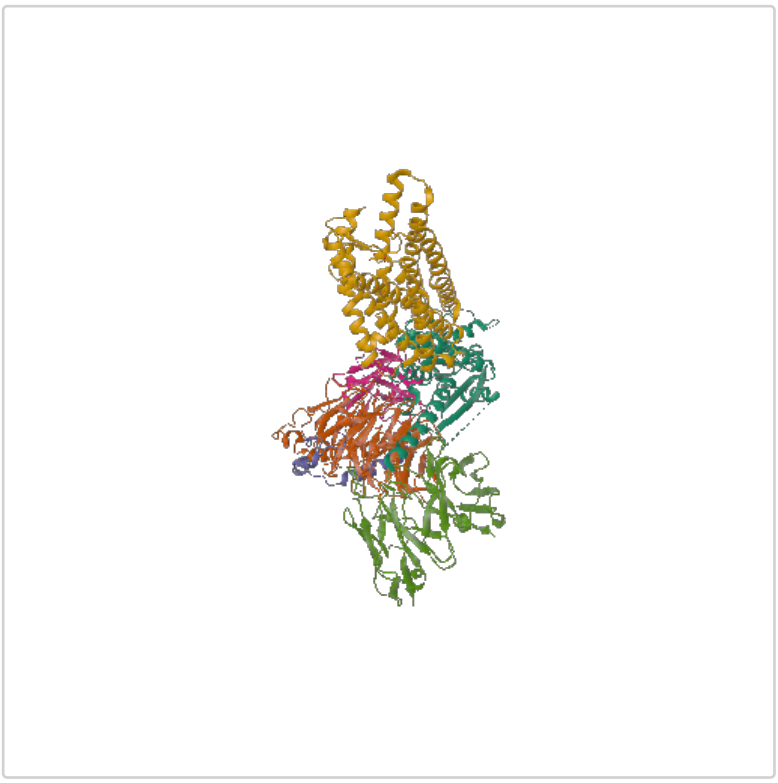

Figure 12: RCSB PDB entry 9M1U rendered as a Cartoon with automatic quality and PNG output.

Structural data source: RCSB PDB / wwPDB, PDB ID `9M1U`, https://doi.org/10.2210/pdb9M1U/pdb[25]. PDB archive data files are available under CC0 1.0. Structure authors: Liu, Zhang, and Xu. Primary citation: Zhang et al. (2026), *The EMBO Journal*, doi:10.1038/s44318-026-00823-y[26].

VII.3 Export For External Tools

The complete 1CRN export example in the Export Commands section generates OBJ, MTL, STL, and PLY bytes from a real BinaryCIF entry and typesets their measured sizes. Typst does not write arbitrary files from a package call; use those byte values inside document logic or expose them through a workflow that may write artifacts outside Typst.

[25] https://doi.org/10.2210/pdb9M1U/pdb
[26] https://doi.org/10.1038/s44318-026-00823-y

Part VIII
Troubleshooting

Symptom	Likely cause	Action
Plugin says Molfig expects bytes.	The input was not bytes, inline string data, or a Typst 0.15+ path value.	For example, pass `path("9R10.pdb")` on Typst 0.15+, or `read("9R10.pdb", encoding: none)` for Typst 0.14-compatible documents.
BinaryCIF reports a missing encoding.	The file is not valid BinaryCIF MessagePack or a column lacks required BinaryCIF encoding metadata.	Check the source file and use `format: "cif"` only for text CIF/mmCIF.
The figure is sparse or missing chains.	Assembly or altLoc selection filtered the structure.	Inspect `molfig.info(...).assemblies` and `alt_locs_info`, then set ⟨`assembly`⟩ and ⟨`alt-loc`⟩ explicitly.
A large assembly compiles slowly.	High tessellation or too much assembly geometry.	Use `quality: "auto"`, ⟨`decimate`⟩, lower ⟨`radial-segments`⟩ and ⟨`linear-segments`⟩, or choose a lighter representation.
SVG parsing fails with `"nodes limit reached"`.	A high-poly mesh, commonly a large spacefill representation, expands to more SVG nodes than Typst accepts.	Use `output-format: "png"`. If vector output is required, reduce ⟨`quality`⟩, ⟨`sphere-detail`⟩, ⟨`linear-segments`⟩, or ⟨`radial-segments`⟩.
Bonds are missing.	The file lacks explicit bonds and inference is disabled.	Leave ⟨`infer-bonds`⟩ as `true`, or inspect `bond_metadata` to see available bond sources.
Surface has triangular fringes or faceted shading.	maquette 0.1.3 does not preserve indexed OBJ/PLY corner normals through smooth shading.	Treat this as a renderer limitation; compare the exported OBJ in a renderer that preserves corner normals.

Rendered view differs from external OBJ inspection.	Different camera, centering, or mesh format.	Pin `⟨center⟩`, `⟨mesh-format⟩`, maquette `⟨config⟩`, and representation quality options.

For reproducible documents, follow the 9R1O Quickstart pattern: use a stable archive identifier, explicit input and mesh formats, a pinned representation, assembly, quality, centering policy, output format, and camera.

Part IX

License And Notices

Molfig package code is distributed under the MIT License. The package also contains third-party material that is covered separately:

Material	Scope	License / notice
`Mol*`	`molfig.wasm` contains Rust behavior and generated reference data derived from `Mol*` parity work.	MIT License; copyright (c) 2017 - now, `Mol*` contributors.
PDB archive data	`examples/data/*.pdb`, `examples/data/*.cif`, `examples/data/*.bcif`, and generated example PDFs.	CC0 1.0 Universal Public Domain Dedication; attribution to PDB IDs and structure authors is encouraged.
PubChem3D examples	`examples/data/*.xyz`, the validation manifest, and generated PDF/PNG.	PubChem-generated conformer coordinates for CIDs 702, 241, 2244, and 2519. PubChem makes generated information available without cost and without restriction; see the recorded NCBI/PubChem policies and attribution.
Typst helper packages	`maquette` for rendering and `mantys` for this manual.	Imported by Typst; not vendored by Molfig.

The package includes `NOTICE.md` and `THIRD_PARTY_NOTICES.md` with the full distribution notice. Example data attributions are also listed in `examples/data/README.md` and `examples/data/ATTRIBUTION.tsv`.

For figures made from PDB archive data, cite the PDB ID and DOI. For example:

Suggested data attribution

```
Structural data source: RCSB PDB / wwPDB, PDB ID 9R1O,
https://doi.org/10.2210/pdb9R1O/pdb.
PDB archive data files are available under CC0 1.0.
```

For a bundled XYZ record, cite its PubChem record and the PubChem service. For example:

Suggested XYZ example attribution

```
Coordinate source: PubChem CID 702 (ethanol), PubChem3D conformer
000002BE00000001, retrieved through PubChem PUG REST on 2026-08-24.
https://pubchem.ncbi.nlm.nih.gov/compound/702
https://doi.org/10.1093/nar/gkae1059
```

When describing Mol* parity or comparing against Mol* output, cite Mol*:

Suggested Mol* citation

```
Sehnal et al. Mol* Viewer: modern web app for 3D visualization and
analysis of large biomolecular structures.
Nucleic Acids Research 49:W431-W437 (2021).
https://doi.org/10.1093/nar/gkab314
```

No endorsement by Mol*, RCSB PDB, wwPDB, structure authors, or data providers is implied.

Part X

Development

The package consists of a Typst wrapper, a Rust WebAssembly plugin, tests, and documentation:

Path	Role
`package/lib.typ`	Public Typst API and maquette bridge.
`wasm-plugin/src/`	Rust parser, Model/Structure/Unit layer, geometry builders, exporters, and Typst plugin ABI.
`wasm-plugin/tests/`	Rust regression tests and Typst contract/integration tests.
`package/docs/documentation.typ`	This Mantys manual.
`package/docs/documentation.pdf`	Compiled reference manual.
`package/molfig.wasm`	Checked-in WebAssembly plugin consumed by Typst.

```
cd wasm-plugin
cargo fmt --check
cargo test
cargo build --release --target wasm32-unknown-unknown
cp target/wasm32-unknown-unknown/release/molfig.wasm ../package/molfig.wasm
cd ../package
just docs
```

Regenerate `package/molfig.wasm` after Rust changes that affect the Typst plugin. Regenerate this PDF after documentation or public API changes.

Part XI

Index

I

M

R

T